Appendix C: IEEE 830 Template

Software Requirements Specification

for

UniLibrary

Version 1.0

Prepared by Szymon Chełmiński

Lodz University of Technology – IFE

03/03/2026

Table of Contents

Revision History

Name	Date	Reason for Changes	Version
Szymon Chełmiński	03/03/2026	Initial version	1.0
Szymon Chełmiński	23/03/2026	Use case diagrams	1.1
Szymon Chełmiński	12/4/2026	UML class	1.2
Szymon Chełmiński	21/4/2026	Added: sequence, state and activity diagrams	1.3
Szymon Chełmiński	03/5/2026	Added github roadmap	1.4

1. Introduction

1.1. Purpose

The purpose of this document is to specify the functional and non-functional requirements for the Library Management System (LMS). This specification is intended for the first version (V.1) of the software, focusing on core lending and cataloging operations.

1.2. Document Conventions

This document follows the IEEE 830 standard. All mandatory requirements are formulated using the keyword "shall" to denote engineering obligations

1.3. Intended Audience and Reading Suggestions

This SRS is intended for developers, project managers, and library staff. Readers should begin with the Product Scope (1.4) before reviewing the specific requirements in Section 5.

1.4. Product Scope

The LMS is a software solution designed to automate the manual processes of a physical library. The system will manage book records, user registrations, and the tracking of borrowed materials to improve operational efficiency

1.5. References

-

2. Overall Description

2.1. Product Perspective

The LMS is a new, self-contained product designed to replace manual paper-based tracking systems.

2.2. Product Features

- User Account Management.
- Book Catalog Search.
- Borrowing and Returning Process.
- Reporting and Analytics.

2.3. User Classes and Characteristics

- **Librarian:** High technical expertise; responsible for managing the catalog and processing loans.
- **Member:** General users who search for and borrow books.

2.4. Operating Environment

The system shall operate as a web-based application compatible with modern browsers (Chrome, Firefox, Safari).

2.5. Design and Implementation Constraints

The system must be developed as a standalone prototype for Lab 1. No specific database technology is mandated at this stage (WHAT, not HOW).

2.6. User Documentation

The system shall be delivered with the following documentation to ensure proper operation:

- **Librarian User Manual:** A comprehensive PDF document providing step-by-step instructions on managing the book catalog, processing loans, and generating reports.

- **Member Quick-Start Guide:** A concise one-page guide for library members explaining how to search the catalog, log in, and check their borrowing status.

- **On-line Help:** Integrated tooltips and a dedicated "Help" section within the Graphical User Interface to assist users with common tasks and troubleshooting.

2.7. Assumptions and Dependencies

The requirements stated in this document are based on the following assumptions and dependencies:

- **Assumption 1:** Users have access to a stable internet connection and a modern web browser (e.g., Google Chrome, Mozilla Firefox, or Safari).

- **Assumption 2:** The library staff will provide the initial book data (titles, authors, ISBNs) in a digital format compatible with the system's import requirements.

- **Dependency 1:** The system depends on a functioning SMTP server to send automated email notifications regarding overdue books and reservation status.

- **Dependency 2:** The system's performance and availability depend on the hosting environment and server infrastructure selected during the implementation phase.

3. System Features

3.1. Book Management

The system allows librarians to maintain an up-to-date digital inventory of all library assets.

3 Lending Services

The system tracks the lifecycle of a book loan from checkout to return.

4. External Interface Requirements

4.1. User Interfaces Overview

4.1. User Interfaces

4.1.1. High-Level Functionality

The system shall provide a Graphical User Interface (GUI) where users interact primarily by clicking buttons and typing into input fields .

User Workflow:

- **Navigation:** Users browse the catalog via a search bar and view results in a list format.

- **Actions:** Librarians add books or register members through dedicated forms, receiving instant text feedback upon submission.

- **Access:** The system uses a standard web-based approach where different features are accessible based on the user's login level.

4.1.2. Interface Standards

- **Layout:** A persistent navigation bar shall be present on every screen, containing a "Help" link and a "Logout" button.

- **Feedback:** Error messages shall appear in a consistent location using red text to clearly indicate invalid inputs.

- **Visuals:** The interface shall follow a clean, professional style with standard buttons for "Save", "Cancel", and "Delete".

4.2. Hardware Interfaces

- **Device Support:** The system shall be compatible with standard computing hardware, including desktop PCs, laptops, and tablets capable of running a modern web browser.
- **Input Peripherals:** The software shall support standard input devices, specifically a keyboard for data entry and a mouse or touch screen for navigation and button interaction.
- **Optional Hardware:** The system shall provide the logical capability to interface with standard USB barcode scanners for reading book ISBNs and member IDs.
- **Communication Protocols:** All interactions between the software and hardware peripherals shall be handled through standard OS-level drivers and HTTP/HTTPS protocols for data transmission.

4.3. Software Interfaces

- **Operating System:** The system shall be platform-independent at the client level, requiring only a modern operating system (Windows, macOS, or Linux) capable of running a web browser.
- **Database Management System (DBMS):** The system shall interface with a relational database to store and retrieve data regarding books, members, and loan transactions. The software shall use standard SQL for data manipulation.
- **External APIs:**

- **Email Service:** The system shall interface with an external SMTP service (such as SendGrid or a local mail server) to dispatch automated notifications to users.

- **ISBN Web Service:** The system shall provide the capability to connect to external book databases (e.g., Google Books API or Open Library API) to automatically fetch book metadata based on ISBN.

- **Web Server:** The application shall be hosted on a standard web server (e.g., Apache or Nginx) and will communicate using the HTTP/HTTPS protocol.

5. System Features/Modules

5.1. Catalog Management

5.1.1 Description and Priority

This feature allows librarians to manage the digital inventory of the library. It is essential for the basic operation of the system. **Priority: High** .

5.1.2 Stimulus/Response Sequences

- **Stimulus:** Librarian enters book details (Title, Author, ISBN) and clicks "Add Book".
- **Response:** The system validates the data, saves it to the database, and displays a "Book Added Successfully" message.
- **Stimulus:** User enters a search query in the search bar.
- **Response:** The system filters the database and displays a list of matching books.

5.1.3 Functional Requirements

- **REQ-1.1:** The system shall store book records including Title, Author, and a unique 13-digit ISBN.
- **REQ-1.2:** The system shall provide a search interface that allows filtering by multiple criteria simultaneously.
- **REQ-1.3:** The system shall prevent the entry of duplicate ISBNs to maintain data integrity.

6. Nonfunctional Requirements

6.1. Performance

- **NF-1.1:** The system shall process and display search results for the library catalog in less than 2.0 seconds under normal load.
- **NF-1.2:** The system shall support up to 50 concurrent users without any degradation in response time.

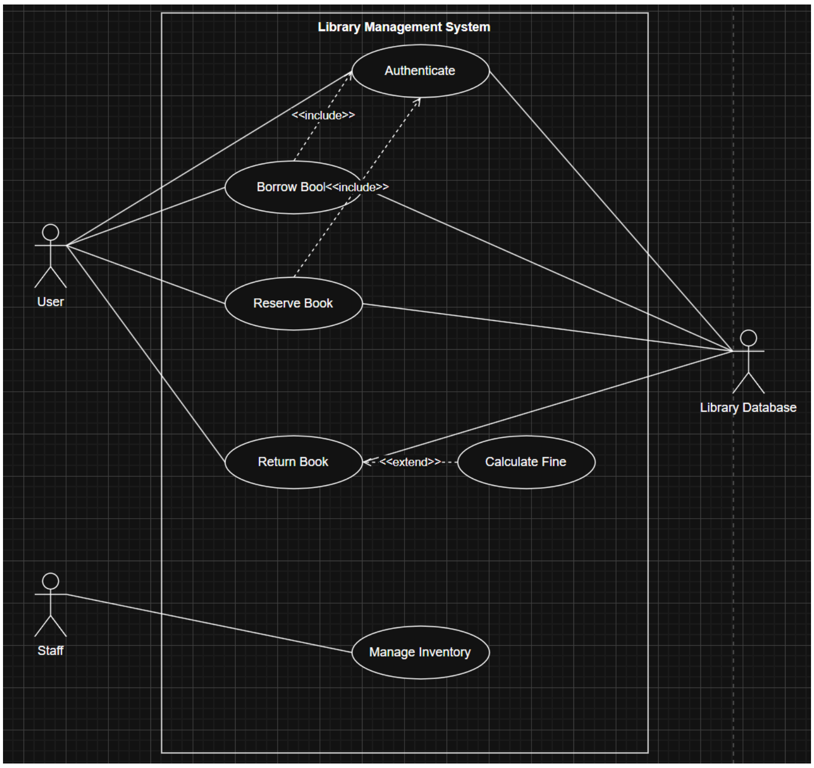
6.2. Security

- **NF-2.1:** The system shall require a unique username and a password of at least 8 characters for librarian authentication.
- **NF-2.2:** All user passwords shall be stored in the database using a secure hashing algorithm such as Argon2 or BCrypt.
- **NF-2.3:** The system shall automatically terminate a user session after 30 minutes of inactivity to prevent unauthorized access.

7. System Models

7.1. Use Case Diagram

The following diagram describes the functional scope of the UniLibrary system and the interactions between the primary actors and the system boundaries.



Description of Interactions:

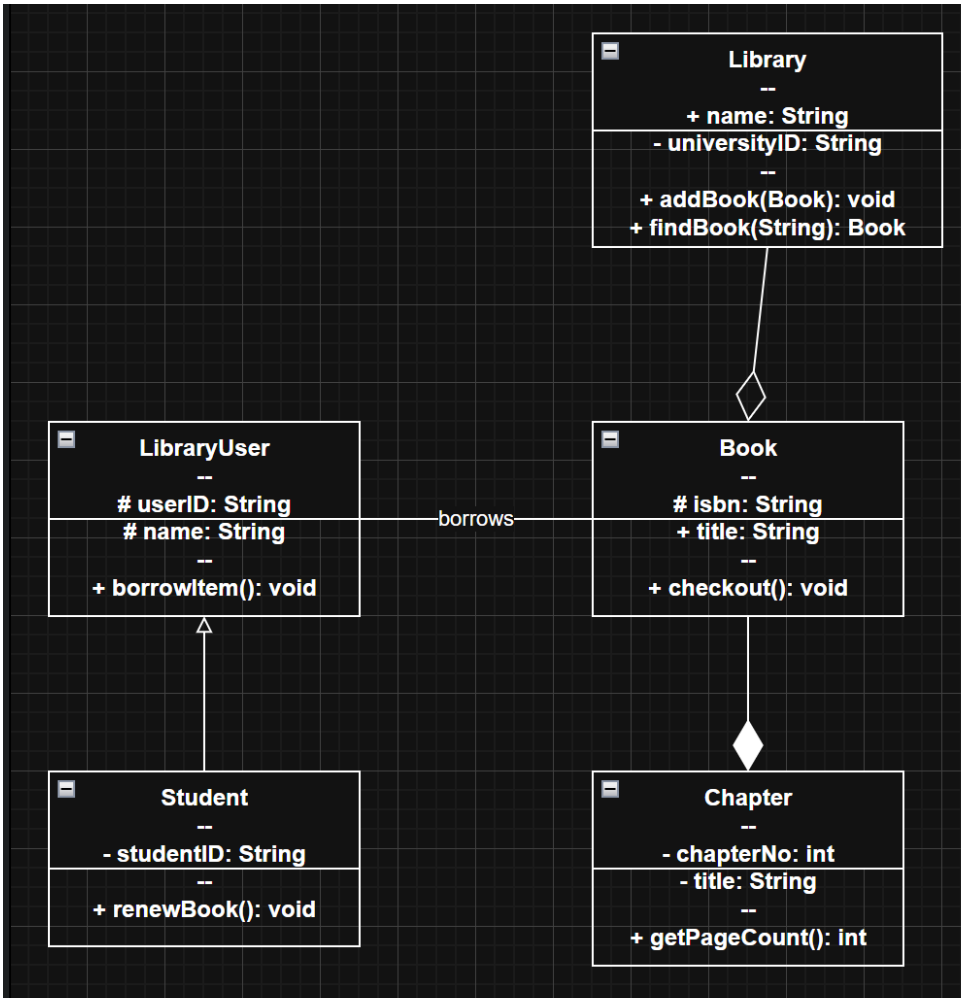
- **Actors:**

- **User: Represents library members who interact with core lending services.**
- **Staff: Represents library personnel responsible for backend operations.**
- **Library Database: An external actor representing the persistent storage system.**

- **Key Relationships:**
    - **Include: Both Borrow Book and Reserve Book use cases include Authenticate, ensuring only authorized users perform these actions.**
    - **Extend: The Calculate Fine use case extends Return Book, occurring only when a returned item is overdue.**
- **Core Functions: Users can manage their book lifecycle (Borrow, Reserve, Return), while Staff members handle Inventory Management.**

7.2. Class Diagram

The class diagram illustrates the static structure and data relationships of the UniLibrary system.



Structural Summary:

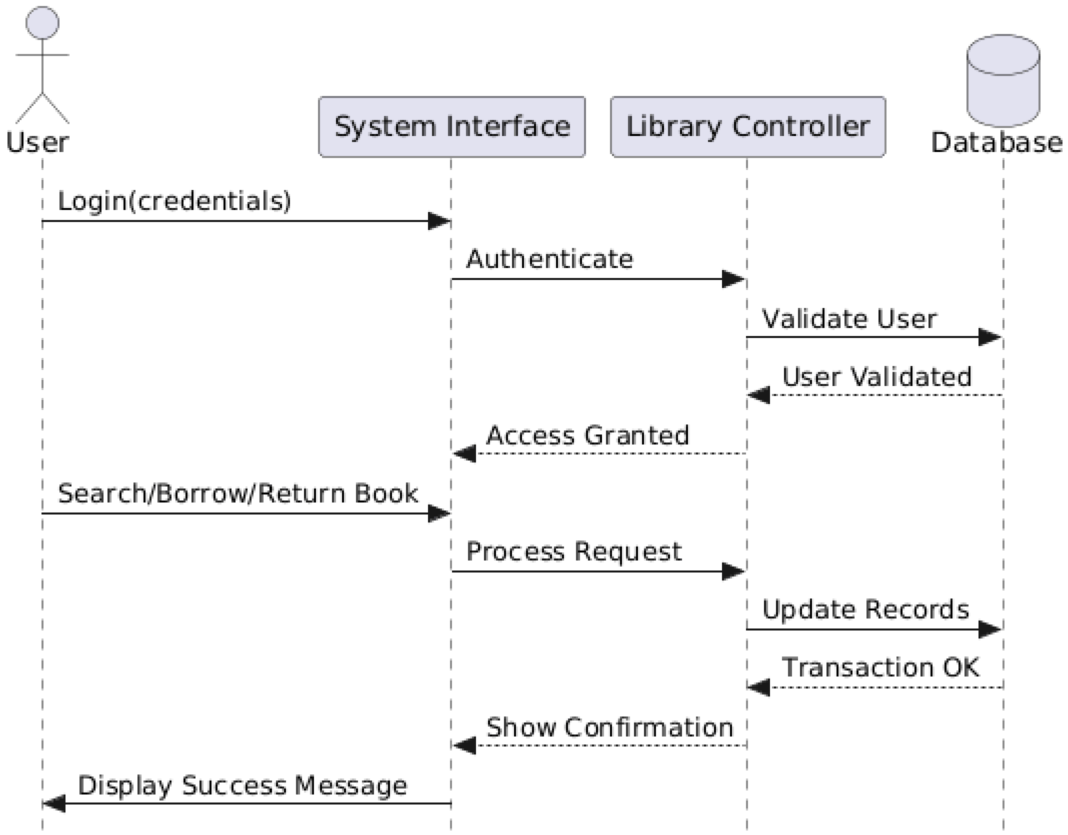
- **Inheritance: Student specializes LibraryUser, adding studentID and renewBook functionality.**
- **Hierarchy: A Library aggregates Book objects, while a Book is composed of multiple Chapters (strict ownership).**

- **Association: The "borrows" relationship connects users to their selected books.**
- **Core Logic: Defined through methods such as addBook(), findBook(), and checkout().**

7.3. Sequence Diagram

This diagram illustrates the dynamic interaction between the user and the system components during authentication and transaction processes.

- Authentication Flow: The User provides credentials via the System Interface, which are then passed to the Library Controller for verification against the Database.
- Transaction Logic: Once access is granted, functional requests (Search, Borrow, or Return) are processed by the controller. The system ensures data integrity by updating records in the database before displaying a final success message to the user.

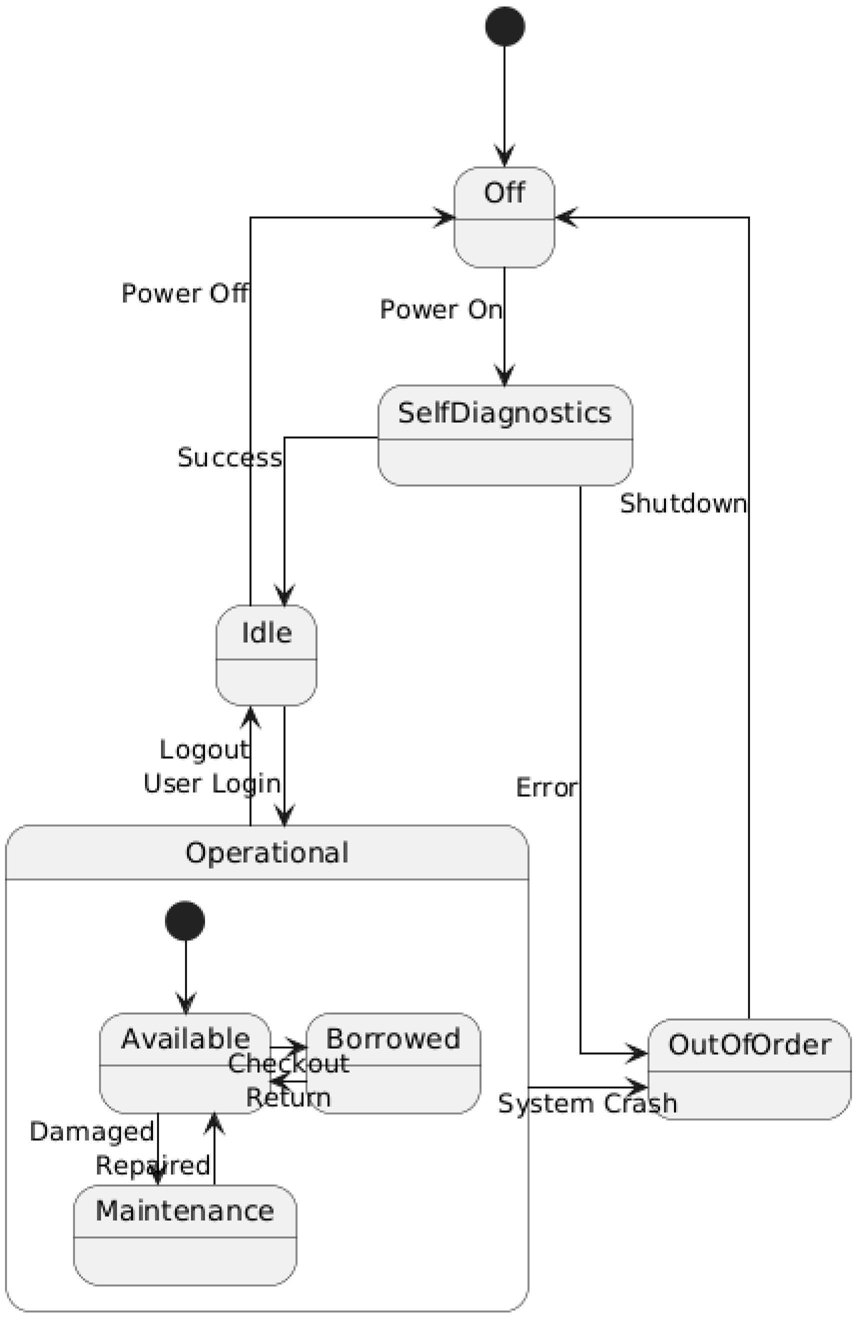


7.4. State Machine Diagram

This diagram defines the operational lifecycle of the system and the various states of book availability.

- System States: The system transitions from Off through SelfDiagnostics to an Idle state. It accounts for error handling, where a system crash or diagnostic failure moves the system into an OutOfOrder state.

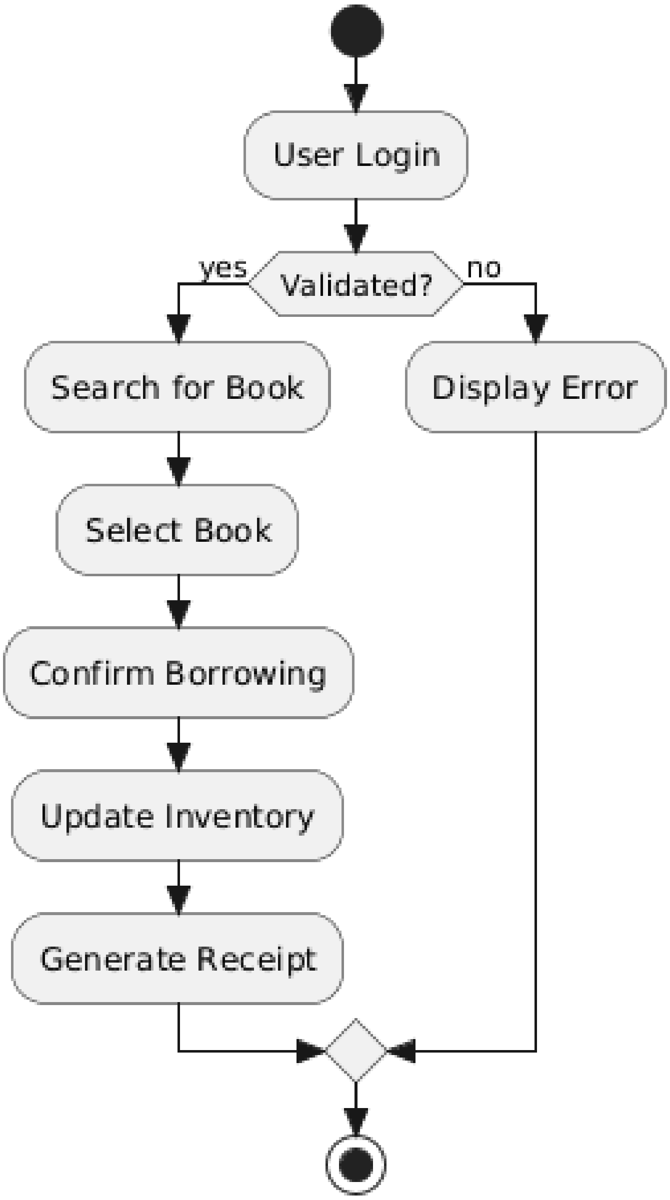
- Book Lifecycle: Within the Operational macro-state, a book moves between Available, Borrowed, and Maintenance. Transitions are triggered by specific actions such as "Checkout", "Return", or "Damaged".



7.5. Activity Diagram

This diagram represents the step-by-step workflow of the book borrowing process.

- Validation Logic: The process begins with a "User Login". A decision node checks if the user is validated; if not, an error is displayed and the process terminates.
- Operational Flow: Upon successful validation, the user follows a linear path: searching for a book, selecting it, and confirming the transaction. The process concludes with an automated inventory update and the generation of a receipt.



8. Project Repository Navigation

This section outlines the structure of the SE_Labs_Portfolio GitHub repository and provides guidance on how to navigate the project artifacts.

8.1. Repository Structure The repository is organized into the following main directories:

- **/Diagrams** Contains the UML models developed across different laboratory sessions.
  - /Lab2_UseCase_Diagrams – Contains Use Case diagrams.
  - /Lab3_Class_Diagrams – Contains UML Class diagrams.
  - /Lab4_Behavioral_Diagrams – Contains Sequence, State Machine, and Activity diagrams.
- **/Project** Contains the core project scope definitions.
  - /Diagrams – Project-specific visual models.
  - readme.txt – A brief overview stating the project's goal to model a modern library management system (collections, members, and borrowing/return processes).
- **/SRS Raports** Serves as the documentation archive. It contains the version history of the IEEE 830 Software Requirements Specification documents (v2, v3, v4) in both .docx and .pdf formats.
- **README.md** The main entry point file providing general information about the repository.

8.2. Navigation Guidelines

- To review the evolution of the system requirements, navigate to the **/SRS Raports** folder and select the latest version of the documentation.
- To inspect specific system models referenced in Section 7 of this document, open the **/Diagrams** directory and navigate to the respective laboratory folder.